
Software Design and Architecture

*Ashenafi Ch.
AASTU, 2022*

Chapter One

Introduction

Discussion

- Software **Design** vs Software **Architecture**
 - » are they the same or different? how?
 - » why we design a software?

Design in SDLC

SOFTWARE DEVELOPMENT LIFE CYCLE



IDEATION

Brainstorming ideas that solve a particular problem faced by target users.



REQUIREMENTS

Interacting with stakeholders and users to collect and document project requirements



DESIGN

Creating the architecture of a software system and its elements.



DEVELOPMENT

Building the software using a programming language by the development team.



TESTING

Evaluating the quality of software with the aim of finding and fixing defects



DEPLOYMENT

Preparing the software to run and operate in a specific environment.



MAINTENANCE

Updating and supporting the software after it has been delivered to the market.

Software Design

- **Design** is a set of activities that need to be performed by the designer in deriving and specifying a solution (***workable solution***) to a problem.
- There may be **more than one possible solution** for a given problem.
- The ***fitness of the solution*** is measured by the **correctness** or **appropriateness** of that solution.
- The purpose of design is to produce a **solution** to a problem:
 - » **The problem:** requirements specification.
 - » **The solution:** your description of how the requirements are to be met.

Cont'd

- The design process consists mainly 4 activities:
 - » Postulate a solution.
 - » Build a model of the solution.
 - » Evaluate the model against original requirements.
 - » Elaborate the model to produce a detailed specification of the solution.
- The nature of the design process
 - » The design process will not be implemented in a single precise sequence.
 - » The design process is iterative in nature.
 - » The design process contains the extensive backtracking.

Cont'd

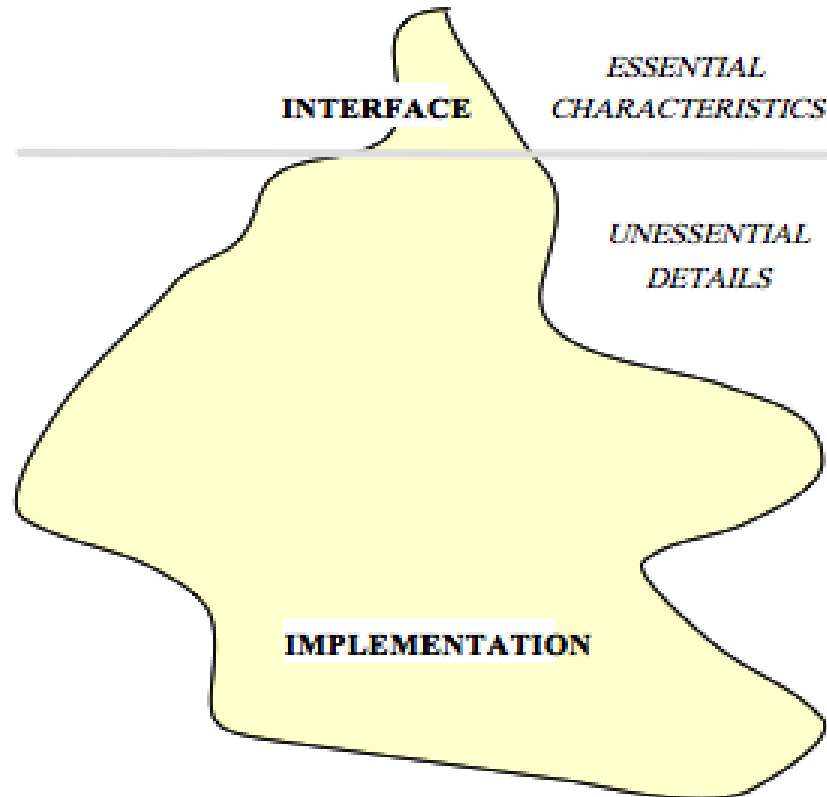
- Design is the **creative process** of describing and transforming a problem into a solution.
- Software design describe:
 - » The system well enough that coders can build it and deploy it without serious problems.
 - » All the parts of the system and how they are to be assembled.
 - » Each part in detail so that it can be coded.
- It consists of a set of documents (diagrams and text).

Design Concepts

- Design concepts provide software designer with basic ideas from which more sophisticated design methods can be applied.
- The fundamental concepts in software design are:
 - » Abstraction
 - » Modularity
 - » Refinement
 - » Architecture

Abstraction

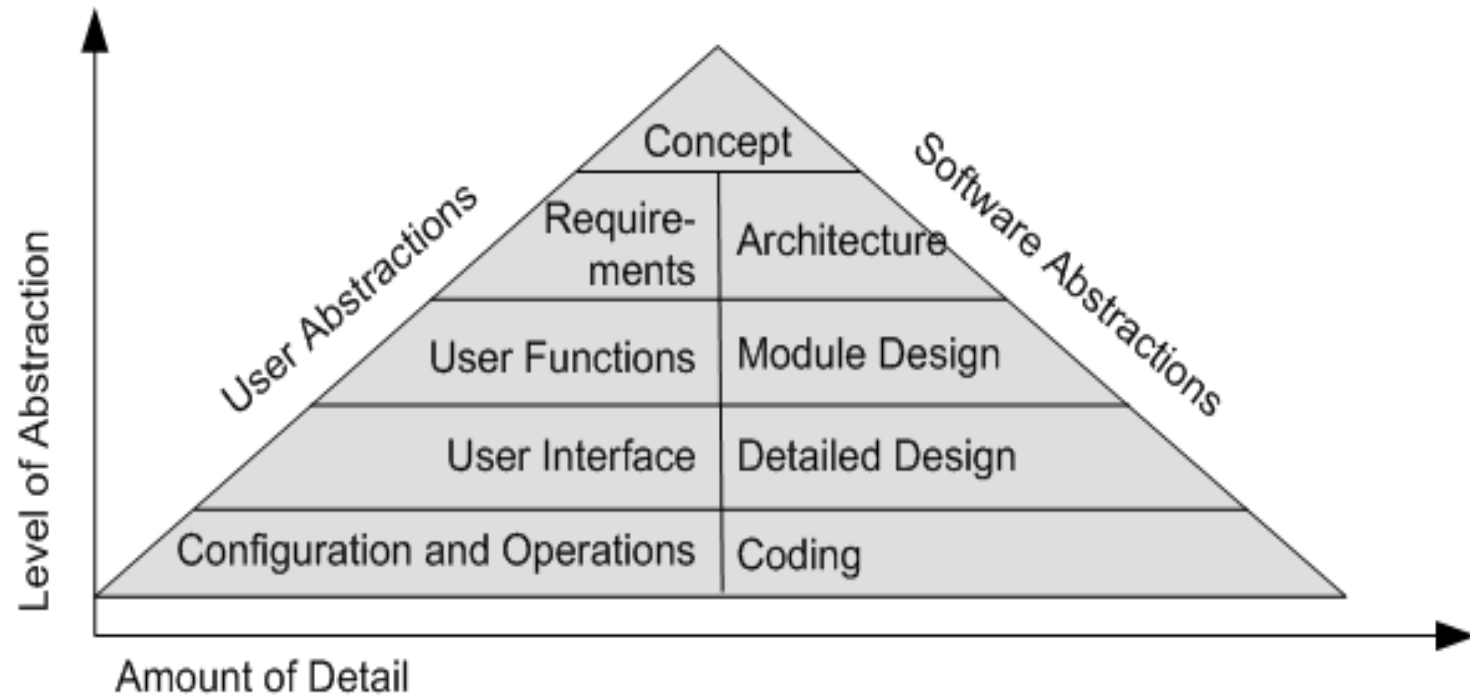
- **Abstraction** allows designers to focus on solving a problem without being concerned about irrelevant lower level details.
- Abstraction manages complexity by emphasizing on essential characteristics and leaving implementation details.



Cont'd

- Abstraction is the selection of a set of concepts to represent a complex whole.
- Abstraction is mostly formed by reducing information content of a concept or a physical phenomenon typically to retain information that is relevant to a particular purpose.
- Each step in the software design process is a refinement in the level of abstraction of the software solution.
- Abstraction methods in Object Oriented Programming (OOP):
 - Encapsulation
 - Information hiding
 - Inheritance
 - Polymorphism and dynamic binding

Cont'd



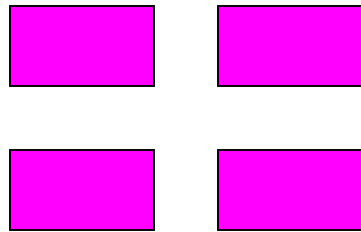
Modularity

- **Modularity** is the degree to which software can be understood by examining its components independently of one another.
- A **module** is a named entity that:
 - » Contains instructions, processing logic, and data structures.
 - » Can be separately compiled and stored in a library.
 - » Can be included in a program.
 - » Module segments can be used by invoking a name and some parameters.
 - » Modules can use other modules.
 - » **Modules:**
 - Any kind of an “independent” software unit.
 - Routines, subroutines (in Java: “methods”; in C: “functions”)
 - Various physical units
 - Files (as in C, C++)
 - Library files (.a is the UNIX convention, DLL in Windows, JAR in Java)

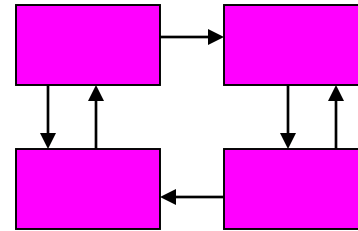
Cont'd

- Partitioning a problem in to sub parts (Modules)
- Quality attributes:
 - » Well-defined: Modules are clearly distinguished.
 - » Separation of concerns: each module has one concern, with minimal overlap between modules.
 - » Loosely coupled.
 - » Cohesive.
- **Cohesion** is a measure of the relative *functional strength* of a module.
- **Coupling** is a measure of the *relative interdependence* among modules.

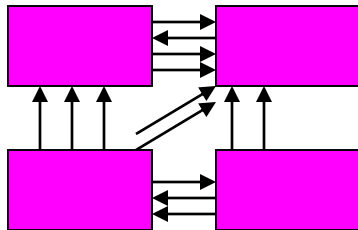
Coupling: Degree of dependence among components



No dependencies



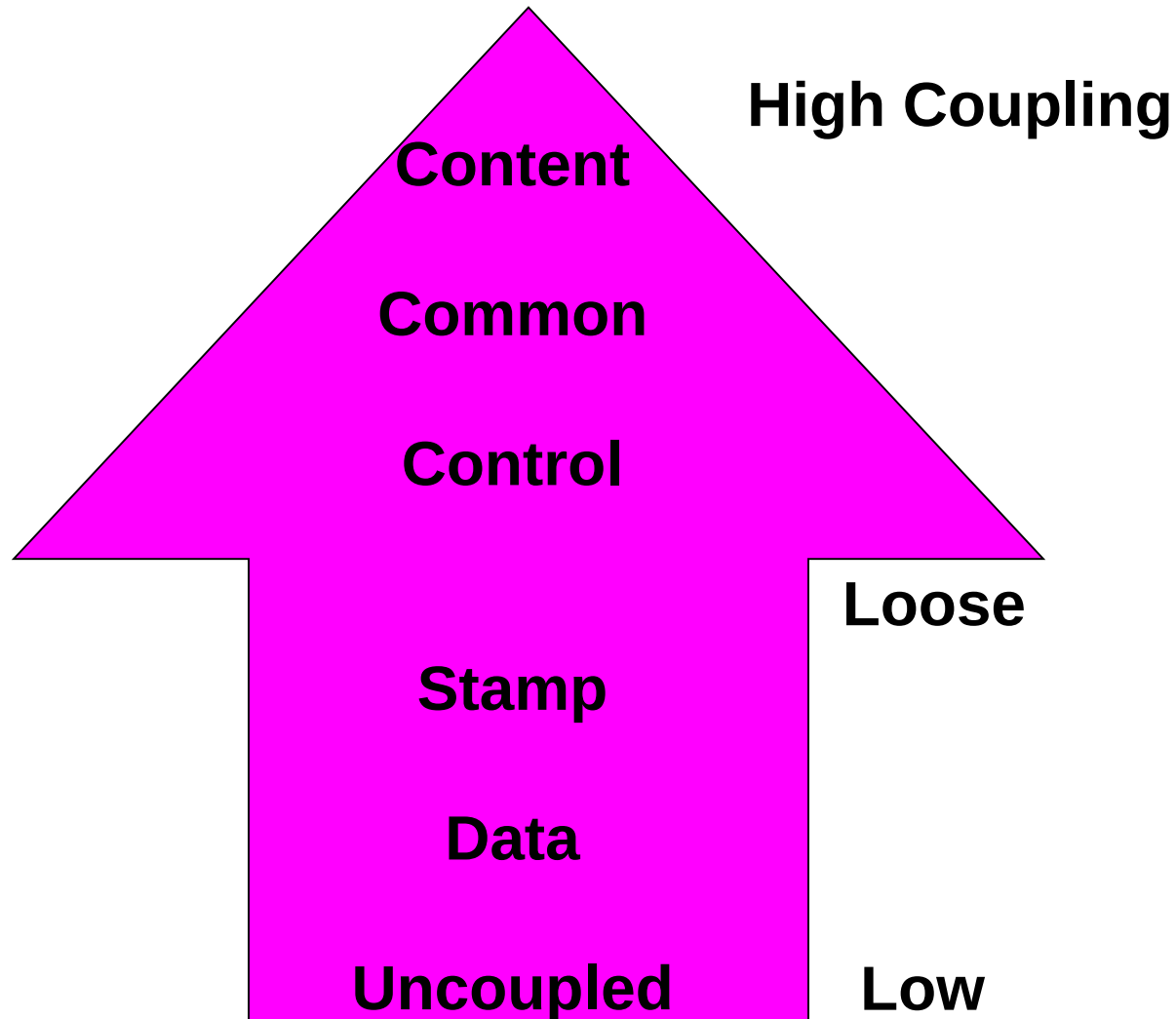
Loosely coupled-some dependencies



Highly coupled-many dependencies

High coupling makes modifying parts of the system difficult, e.g., modifying a component affects all the components to which the component is connected.

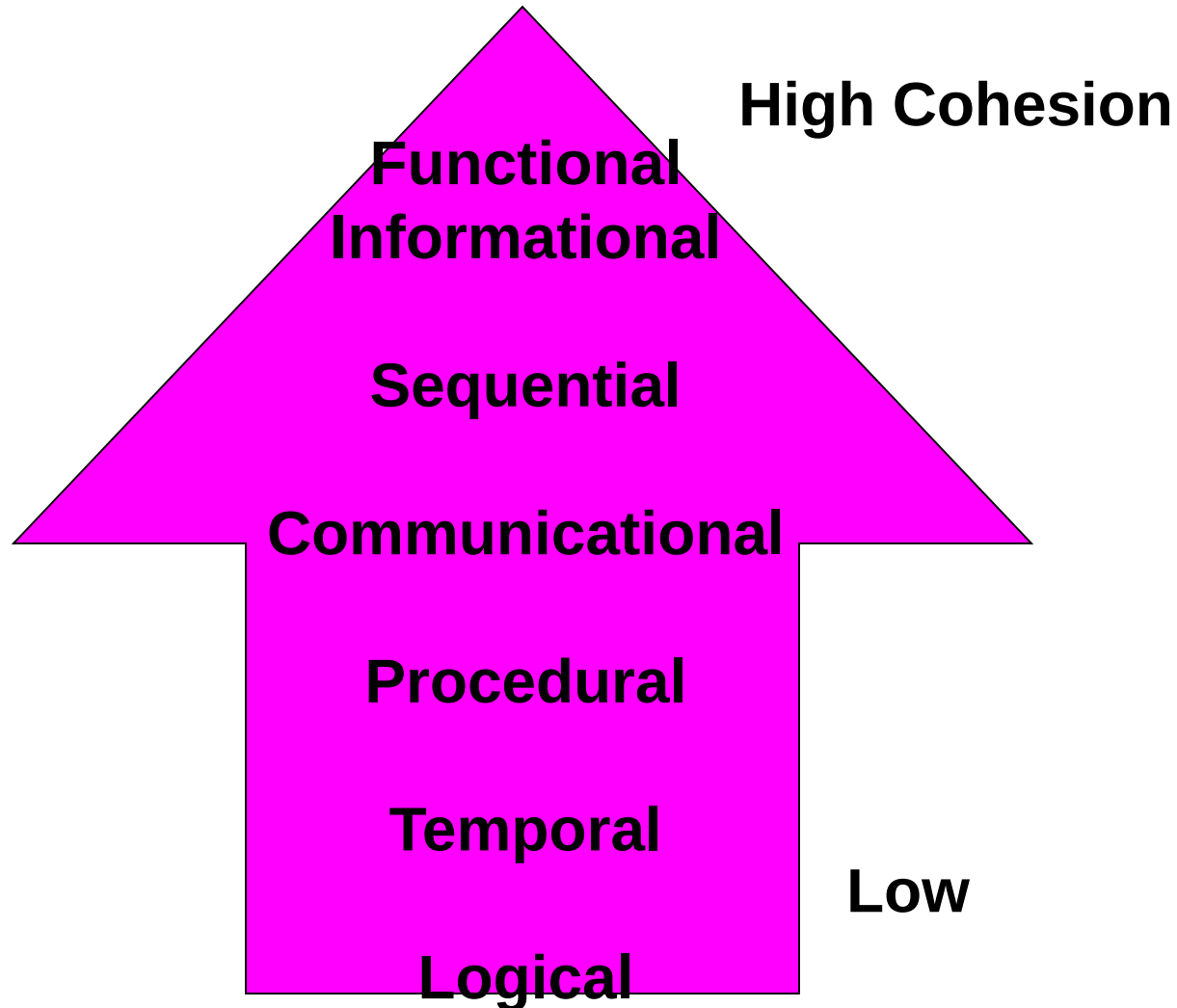
Range of Coupling



Cohesion

- The degree to which all elements of a component are directed towards a single task and all elements directed towards that task are contained in a single component.
- Internal glue with which component is constructed.
- All elements of component are directed toward and essential for performing the same task.
- High is good.

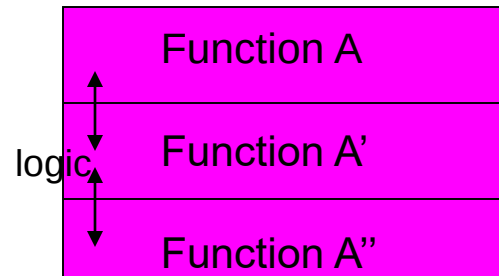
Range of Cohesion



Examples of Cohesion-1

Function A	
Function B	Function C
Function D	Function E

Coincidental
Parts unrelated



Logical
Similar functions

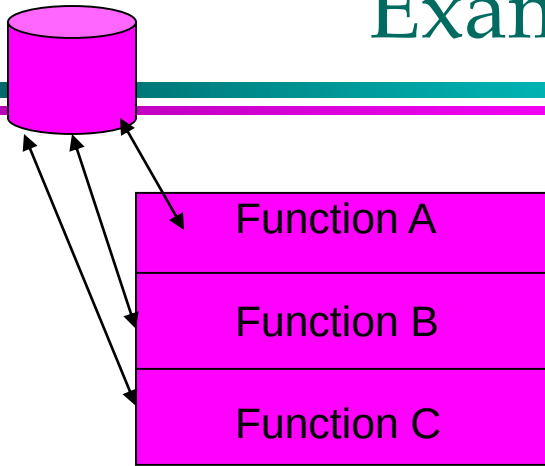
Time t_0
Time $t_0 + X$
Time $t_0 + 2X$

Temporal
Related by time

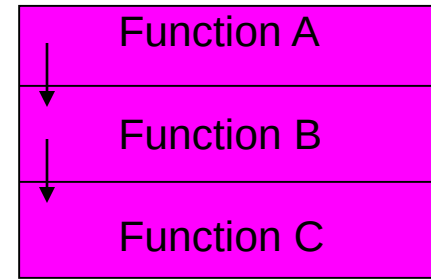
Function A
Function B
Function C

Procedural
Related by order of functions

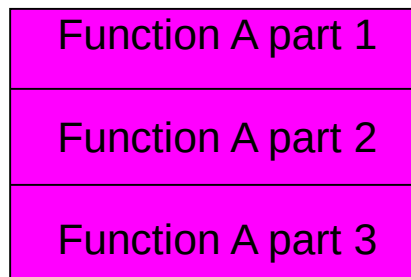
Examples of Cohesion-2



Communicational
Access same data



Sequential
Output of one is input to another



Functional
Sequential with complete, related functions

Group Assignment

- Form a group of **two members**.
- Review all the **types** of **Coupling** and **Cohesion** mentioned in the previous slides.
- Prepare a report from **3-5 pages**.
- Due date: **30/11/2022**

Cont'd

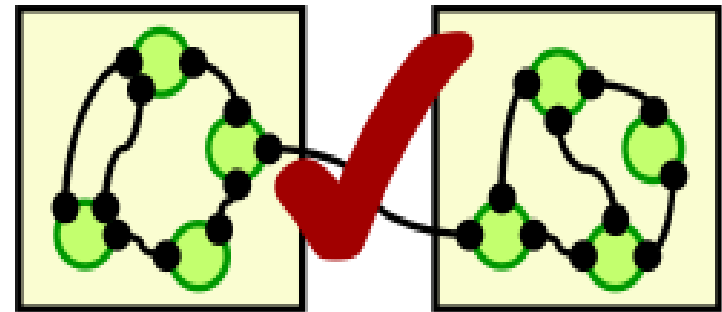
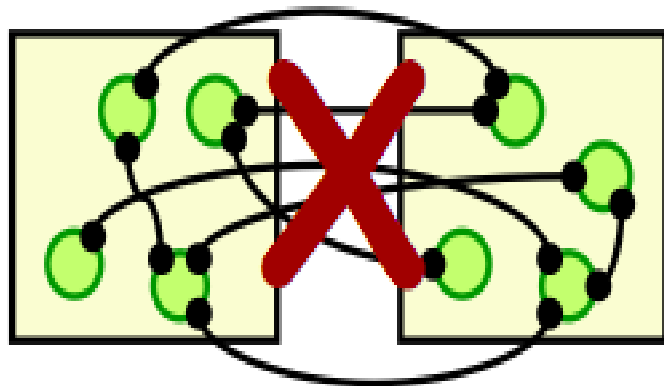
- **A good architecture:**

- » Minimizes coupling between modules:

- Goal: modules don't need to know much about one another to interact.
- Low coupling makes future change easier.

- » Maximizes the cohesion of each module:

- Goal: the contents of each module are strongly inter-related
- High cohesion makes a module easier to understand.



Cont'd

- **Advantages of Modularity**
 - » Comprehensibility/Understandability.
 - » Parallel development of different modules is possible.
 - » Quality of design since it will be easier to:
 - Test
 - Verify and validate
 - Measure reliability
 - Maintainability(It will be easier to):
 - Locate faults and correct them
 - Enhance
 - Reuse modules for different design work

Refinement

- Refinement is the process of elaboration where the designer provides successively more detail for each design component.
- It begins with a statement of function (or description of information) that is defined at a high level of abstraction.
 - » Decompose design decision top-down to elementary level.
 - » Isolate design aspects that are not independent.
 - » Add details incrementally in each step.
 - » Postpone decisions relating to detailed representations.
 - » Continually demonstrate that each refinement step is a correct expansion of previous steps.

Cont'd

- Abstraction and refinement are complementary concepts.
 - » Abstraction enables a designer to specify procedure and data and yet do not focus on low-level details.
 - » Refinement helps the designer to reveal low-level details as design progresses.
 - » Both concepts aid designers in creating a complete design model as the design evolves.

Architecture

- It provides an overall structure of the software and the ways in which that structure provides conceptual integrity for the system.
- It serves as a framework from which more detailed design activities are conducted.

Common Design Principles

- Basic design principles help software designers to manage the design process.
- When these design principles are properly applied, the software engineer creates a design that shows both external and internal quality factors.
- » **External Quality Factors**
 - These are properties of the software that can be observed by users
 - E.g. speed, usability, reliability, etc
- » **Internal Quality Factors:**
 - Properties of the software seen from the technical perspective (the operations of the application)

Cont'd

1. **Consideration of alternative approaches**
 - » Decision on the approaches to follow must be made based on
 - the requirements of the problem, the resources available to do the job, and design concepts
2. **The design should be traceable to the analysis model.**
 - » it is necessary to have a means for tracking how requirements have been satisfied by the design model.
3. **The design should not reinvent the wheel.**
 - » Design time should be invested in representing truly new ideas and integrating those patterns that already exist.
4. **Reduced intellectual gap between software and the problem**
 - » The structure of the software design should (whenever possible) reflect the structure of the problem domain.

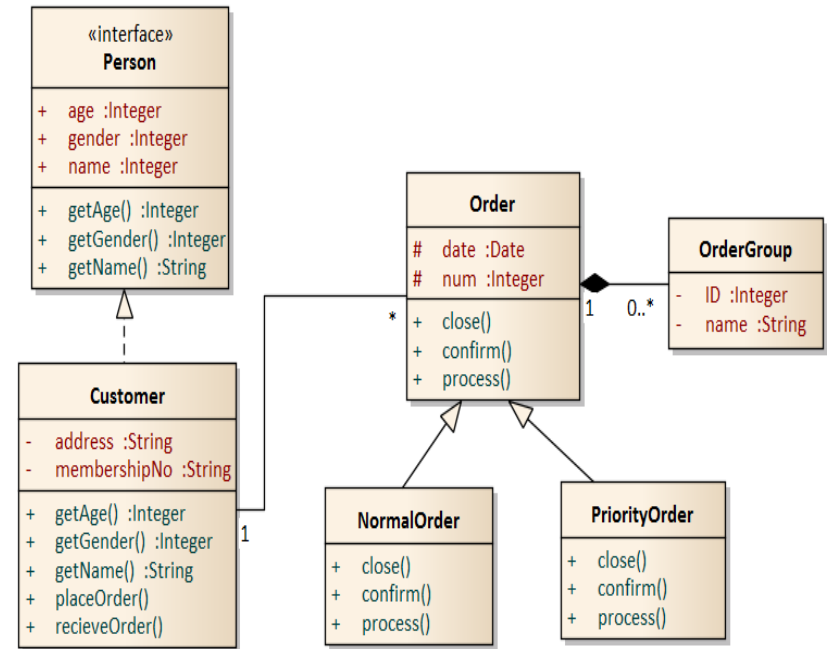
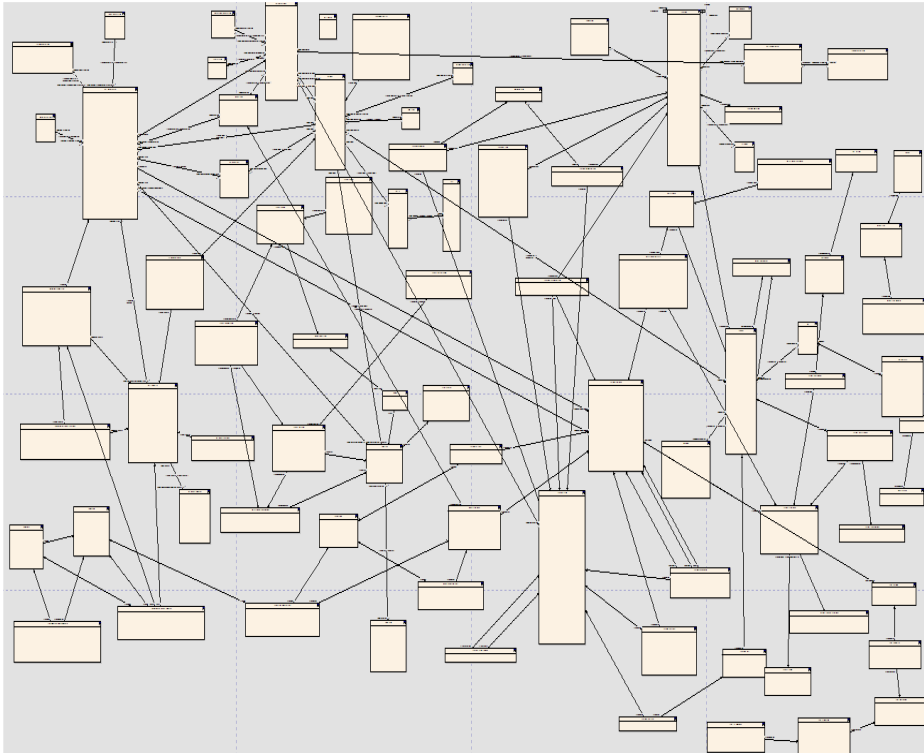
Cont'd

5. **The design should exhibit uniformity and integration**
 - » A design is uniform if it appears that one person developed the entire thing.
6. **The design should be structured to accommodate change.**
 - » The design should allow maintainability
7. **Quality assessment of the design during the development process**
 - » A variety of design concepts and design measures are available to assist the designer in assessing quality.
8. **The design should be structured to degrade gently (perform at a lower quality level), even when erroneous data, events, or operating conditions are encountered.**

Cont'd

9. **Design is not coding, coding is not design.**
 - » Even when detailed procedural designs are created for program components, the level of abstraction of the design model is higher than source code.
10. **The design should be reviewed to minimize conceptual (semantic) errors.**
 - » A design team should ensure that major conceptual elements of the design (omissions, ambiguity, inconsistency) have been addressed before worrying about the syntax of the design model.

Keep it Simple (KIS)



Goals of Software Design

- Correctness
- Robustness
- Flexibility
- Reusability
- Efficiency
- Reliability
- Usability

Cont'd

- **Correctness**

- » Software design **must satisfy the requirements for the application**
- » The software designed must perform what it is intended to do

- **Robustness**

- » A design or implementation is robust **if it is able to handle miscellaneous and unusual conditions** such as bad data, user error, programmer error, and environmental conditions.

Cont'd

Flexibility

- Requirements of an application **can change in many ways**.
- *Design should be flexible to accommodate these changes.*

Reusability

- The trend in software is to reuse parts among applications
 - » **Good designs are more easily modified and reused**

Efficiency

- » It refers to the **use of available machine cycles and memory**
- » Efficiency conflicts with other goals of software design

Cont'd

Reliability

- » An application is reliable if it is relatively defect free.

Usability

- » An application has high usability if users find it easy to use.

What is Software Architecture?

- The software architecture of a program or computing system is the **structure** or structures of the system, which comprise software **elements**, the **externally visible properties** of those elements, and the **relationships** among them.
- The exact structures to consider and the ways to represent them vary according to engineering goals.

Cont'd

- A software architecture is an abstraction of a system.
 - » Architecture defines elements and how they interact.
 - » Architecture suppresses purely local information about elements; private details are not architectural.
- Externally-visible properties of elements are assumptions that one elements can make about another:
 - » provided services, required services, performance characteristics, fault handling, resource usage

Cont'd

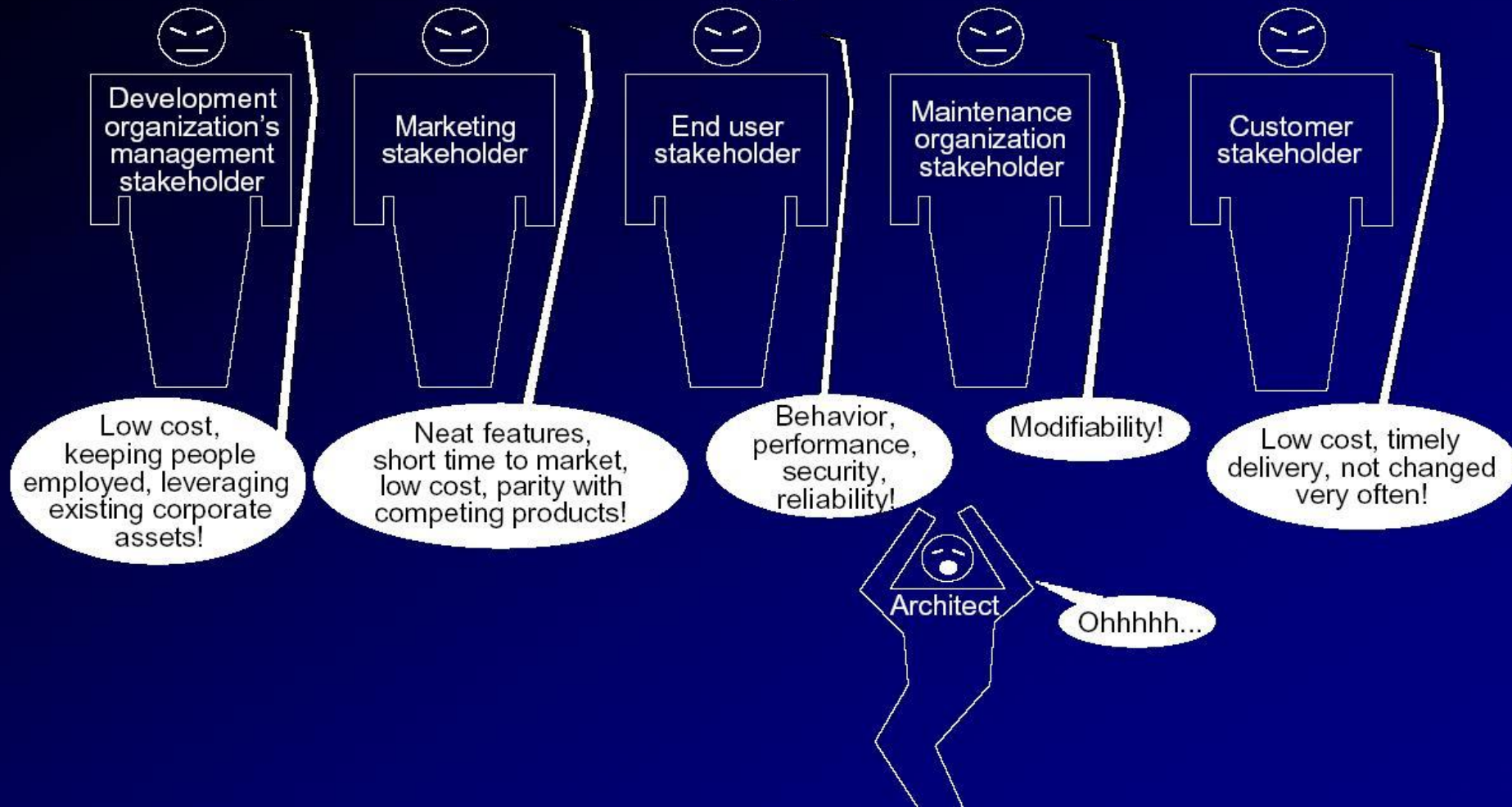
- Every system *has* an architecture.
 - » Every system is composed of elements and there are relationships among them.
 - » In the simplest case, a system is composed of a single elements, related only to itself.
- Just having an architecture is different from having an architecture that is known to everyone:
 - » The architecture versus specification of the architecture
- If you don't explicitly develop an architecture, you will get one anyway – *and you might not like what you get.*

Cont'd

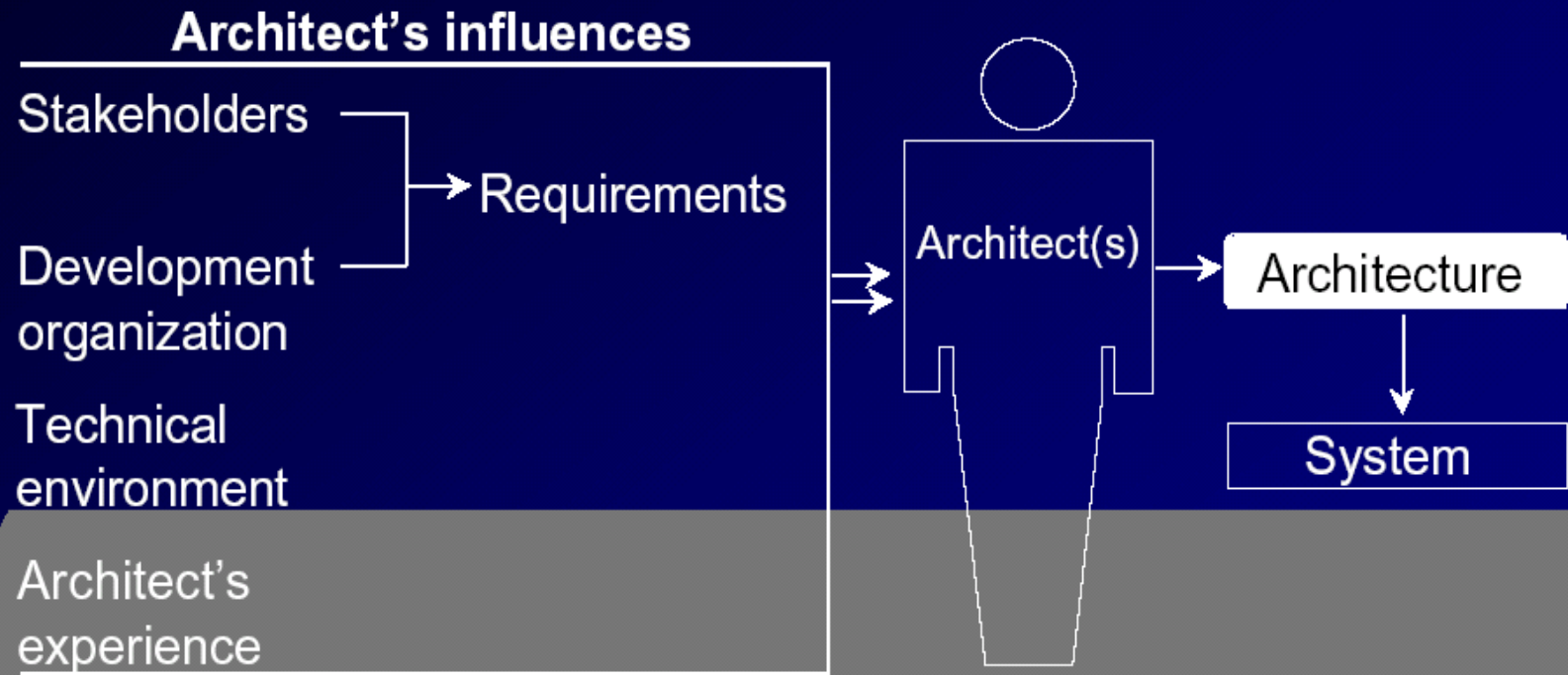
- This means that box-and-line drawings alone are *not* architectures; but they are just a starting point.
 - » You might *imagine* the behavior of a box labeled “database” or “executive” -- but that’s all
 - » You need to add specifications and properties.
- Systems have many structures (views).
 - » No single structure can be *the* architecture.
 - » The set of candidate structures is not fixed or prescribed: choose whatever is useful for analysis, communication, or understanding.
 - » An architecture is a very complicated construct -- too complicated to be seen all at once.
 - » *Views* are a way to manage complexity.

Who Influences Software Architecture?

Stakeholders of a System



Cont'd



Importance of Software Architecture

- “If a project has not achieved a system architecture, including its rationale, the **project should not proceed** to full-scale system development. Specifying the architecture as a deliverable enables its use throughout the **development and maintenance** process.”

Barry Boehm

Architecture as Communication Vehicle

- Architecture provides a common frame of reference in which competing interests may be exposed and negotiated.
 - » negotiating requirements with users and other stakeholders
 - » keeping the customer informed of progress and cost
 - » implementing management decisions and allocations

Architecture Constrains Implementation

- An architecture defines constraints on an implementation.
 - » Architectures are descriptive and prescriptive
 - descriptive for communication
 - prescriptive for design and implementation
 - » Global resource allocation decisions constrain implementations of individual components
 - » System tradeoffs regarding quality attributes are in the architectural realm.

Architecture Organizes the Development Project

- The architecture influences the organizational structure for development/maintenance efforts. Examples include
 - » division into teams
 - » units for budgeting, planning
 - » basis of work breakdown structure
 - » organization of documentation
 - » organization of CM libraries
 - » basis of integration
 - » basis of test plans, testing
 - » basis of maintenance
 - » Incremental deployment

Architecture Permits/Precludes Achievement of Quality Attributes

- For example

<i>If you desire</i>	<i>Examine</i>
performance	inter-component communication
modifiability	component responsibilities
security	inter-component communication, specialized components (e. g., kernels)
scalability	localization of resources
ability to subset	inter-component usage
reusability	inter-component coupling

Architecture is Key to Managing Change

- An architecture helps reason about and manage change.
 - » important since $\approx 80\%$ of effort in systems occurs *after* deployment
- Architecture divides all changes into three classes:
 - » **local**: modifying a single component
 - » **non-local**: modifying several components
 - » **architectural**: modifying the gross system topology, communication, and coordination mechanisms
- A “good” architecture is one in which the most likely changes are also the easiest to make.

Architecture is Basis for Incremental Development

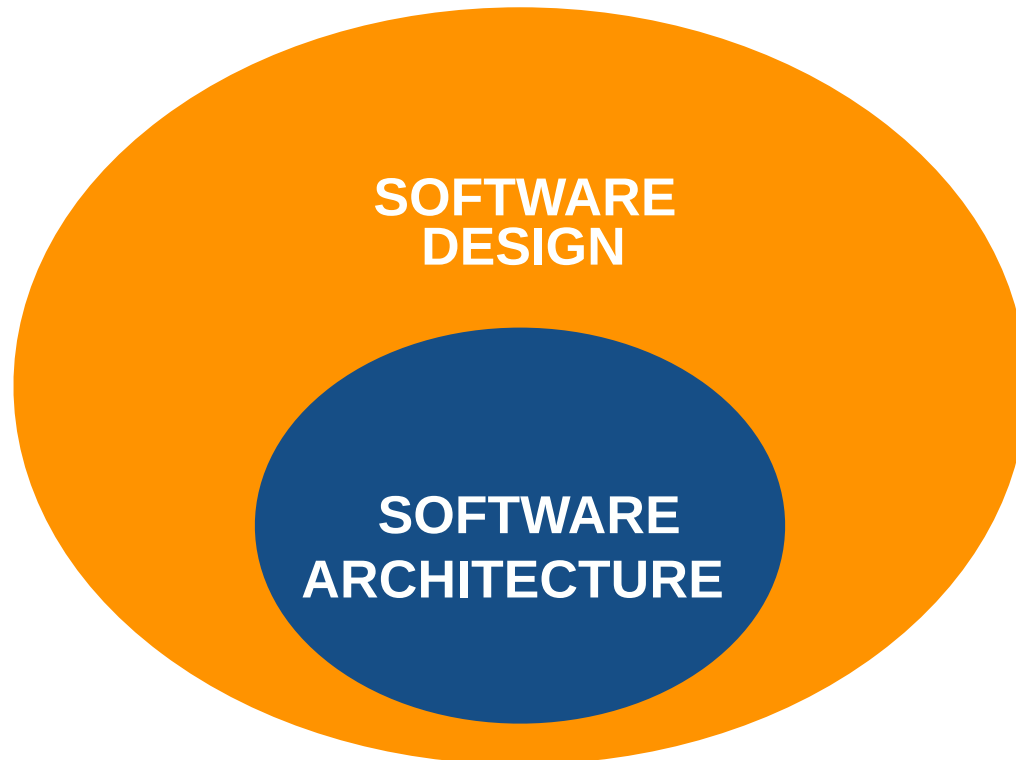
- An architecture helps with evolutionary prototyping and incremental delivery.
 - » Architecture serves as a skeletal framework into which components can be plugged.
 - » By segregating functionality into appropriate components, experimentation is easier.
 - » Risky elements of the system can be identified via the architecture and mitigated with targeted prototypes.

Architecture is a Reusable Model

- An architecture is an abstraction: enables a one-to-many mapping (one architecture, many systems).
- Systems can be built from large, externally developed components that are tied together via architecture.
- Architecture is the basis for product (system) commonality.
- Entire *software product lines* can share a single architecture.

Architecture vs Design

- All architecture is design.
- NOT all design is architecture.
- Architect draws the boundary between architectural and non-architectural design.



Summary: Software Architecture

- Represents the structure of the software.
- Expressed using certain views.
- Includes principles behind the design and evolution of the software.
- Essential characteristics of architecture:
 - » High level view
 - » Realize ALL the use case scenarios
 - » Present a systemic view to all stakeholders